SENTINEL TWIN: DIGITAL TWIN AND ENDPOINT DETECTION

A PROJECT REPORT

Submitted by

HEMANTH KUMAR A. **2116231901010**

SIVARANGINI Y. **2116231901051**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**SENTINEL TWIN: DIGITAL TWIN AND ENDPOINT DETECTION**" is the bonafide work of "**HEMANTH KUMAR A. (2116231901010), SIVARANGINI Y. (2116231901051)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

6/10/26

SIGNATURE
Dr. P. Tamilselvi, M.E., Ph.D.,
HEAD OF THE DEPARTMENT
PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

V. Bala 6/x/26

SIGNATURE
Dr. Balasubramanian V.
SUPERVISOR
PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on **06/10/2026**

Internal Examiner

6/10/26
External Examiner

ABSTRACT

Sentinel Twin is a local endpoint monitoring system that compares the present behaviour of a Windows computer with a behavioural reference learned from the same machine. A Python agent collects process details, CPU and memory usage, thread activity and file changes in a watched folder every 1.5 seconds and converts each reading into a ten-feature vector. Three detectors work on this vector. A digital twin built on Isolation Forest measures how far the reading has moved from the learned baseline, a Random Forest classifier gives a learned score, and a rule engine checks selected process names and bursts of activity. The three scores are combined with fixed weights into a single risk value that is placed in one of four severity bands.

A Flask dashboard with an interactive 3D view shows the live readings, the contribution of each detector, the features that changed most, and a searchable alert feed. The user can pause monitoring, reset the baseline, filter alerts and export an incident as JSON. Rules that match a reviewed list are linked to possible MITRE ATT&CK techniques, and the alert records which evidence is still missing, so a name-only match stays a hypothesis. Phase I covers the problem study, literature survey, requirements, system design, the working prototype and an analysis of the dashboard results. The Random Forest is trained on synthetic data, so its holdout scores show that the pipeline works and do not measure real malware detection. Phase II will add event-based collection, independently labelled endpoint data and durable incident storage.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S. N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi M.E., Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Dr. Balasubramanian V, Professor,** for his encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

We also acknowledge the open-source communities behind Python, Flask, scikit-learn, psutil, NumPy, pytest and the other tools used in this project.

HEMANTH KUMAR A. **2116231901010**

SIVARANGINI Y. **2116231901051**

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	2
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS	3
	1.6 SDG ALIGNMENT	4
2	**LITERATURE REVIEW**	**5**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	5
	2.1.1 ALERT QUALITY AND ANALYST WORKLOAD	6
	2.1.2 PROVENANCE AND EVENT CORRELATION	6
	2.1.3 MITRE ATT&CK IN DETECTION ENGINEERING	7
	2.1.4 MACHINE LEARNING AND EVALUATION BIAS	7
	2.1.5 DIGITAL TWINS AND BEHAVIOURAL BASELINES	8
	2.2 RESEARCH GAPS	8
	2.3 PROPOSED SYSTEM	9
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	9
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	9
	2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM	10
3	**SYSTEM DESIGN**	**12**
	3.1 SYSTEM ARCHITECTURE	12
	3.2 MODULE DESIGN	15
	3.2.1 COLLECTION MODULE	15
	3.2.2 FEATURE PREPARATION MODULE	15
	3.2.3 DIGITAL TWIN MODULE	15
	3.2.4 CLASSIFIER AND RULE MODULES	16
	3.2.5 DECISION AND MAPPING MODULE	16
	3.2.6 DASHBOARD MODULE	16
	3.3 UML DIAGRAMS	16
	3.3.1 USE CASE DIAGRAM	16
	3.3.2 WORKFLOW DIAGRAM	18
	3.3.3 DATA FLOW DIAGRAMS	20
	3.3.4 SEQUENCE DIAGRAM	21

CHAPTER NO.	TITLE	PAGE NO.
	3.3.5 STATE DIAGRAM	22
	3.3.6 CLASS DIAGRAM	23
	3.3.7 DEPLOYMENT DIAGRAM	24
	3.4 THREAT MODEL AND ATTACK SURFACE	25
	3.5 SECURITY CONTROLS	26
4	**SYSTEM DESCRIPTION**	**28**
	4.1 DATA COLLECTION	28
	4.2 DATA PREPROCESSING	28
	4.2.1 ALGORITHMS	29
	4.2.2 RESULTS AND DISCUSSION	33
5	**CONCLUSION**	**36**
	REFERENCES	**38**
	APPENDIX	**41**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	PROJECT OBJECTIVES	3
1.2	SDG ALIGNMENT	4
2.1	DETECTION APPROACHES CONSIDERED	5
2.2	RESEARCH GAPS AND PROJECT RESPONSE	8
2.3	FUNCTIONAL REQUIREMENTS	10
2.4	COMPARISON OF EXISTING AND PROPOSED SYSTEM	11
2.5	FEASIBILITY STUDY	11
3.1	TECHNOLOGY STACK	14
3.2	HARDWARE AND SOFTWARE REQUIREMENTS	14
3.3	MODULES AND RESPONSIBILITIES	15
3.4	USE CASE DESCRIPTIONS	18
3.5	MONITORING STATES	23
3.6	THREATS AND THEIR IMPACT	26
3.7	SECURITY CONTROLS	27
4.1	COLLECTED INFORMATION	28
4.2	DEFAULT SCORING AND MONITORING SETTINGS	31
5.1	PHASE II WORK PLAN	37

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
2.1	CONVENTIONAL ALERTING VS PROPOSED SYSTEM	9
3.1	SYSTEM ARCHITECTURE	13
3.2	USE CASE DIAGRAM OF SENTINEL TWIN	17
3.3	SYSTEM WORKFLOW	19
3.4	LEVEL 0 DATA FLOW DIAGRAM	20
3.5	LEVEL 1 DATA FLOW DIAGRAM	21
3.6	SEQUENCE DIAGRAM OF A MONITORING SESSION	22
3.7	STATE DIAGRAM OF THE MONITORING AGENT	23
3.8	CLASS DIAGRAM OF THE DATA MODEL	24
3.9	DEPLOYMENT DIAGRAM	25
3.10	TRUST BOUNDARIES AND THREAT LOCATIONS	26
4.1	RISK FUSION AND SEVERITY BANDS	31
4.2	EVIDENCE CHECKS FOR ATT&CK MAPPING	32
4.3	MONITORING DASHBOARD OVERVIEW	33
4.4	DETECTOR SCORES AND RISK TREND	34
4.5	FEATURE EXPLANATIONS AND ALERT FEED	35

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Every organisation, from a college computer lab to a large company, depends on its endpoints: the laptops and desktops where people read mail, write code, browse the web and keep their files. These machines are also where an attack finally has to do its work. Whatever route an intruder takes, sooner or later a program has to start on some computer, use its processor and memory, and read or change files there. That is why watching the endpoint has become one of the main layers of defence.

The difficulty is that a healthy computer is never quiet. A browser update, a large compile or a backup job can push CPU use and file activity well above the usual level for several minutes, and none of these is a threat. A monitor that treats every spike as suspicious soon produces so many warnings that people stop reading them. To be useful, a monitor has to know what normal looks like on the particular machine it is watching, and it has to be able to say plainly why a reading looks wrong.

Sentinel Twin was built around that idea. It keeps a learned picture of how one endpoint usually behaves, which this report calls its digital twin, and compares every new reading with it. The twin is a statistical model of normal activity, not a copy of the operating system. Alongside it sit a Random Forest classifier and a small set of explicit rules, so each decision draws on three different kinds of evidence, and the three scores stay visible on the dashboard instead of disappearing into a single number.

The project is about observation and explanation. The dashboard shows what was collected, how each detector scored it and which measurements moved the most, and every alert keeps that information so that it can be reviewed later. The current version does not block processes or quarantine files on its own. Those actions need stronger evidence and a clear response policy, so they are left for later work.

Motivation

Alahmadi et al. [1] found that analysts in security operations centres spend much of their day on alarms that turn out to be harmless, and that checking each one by hand is slow

because the alarm rarely carries enough context. Students and small teams face the same problem on a smaller scale, usually without access to a commercial endpoint detection product. A lightweight monitor that runs on one machine, learns its normal behaviour and explains its alerts is a practical way to study how such tools ought to behave.

1.2 PROBLEM STATEMENT

Many endpoint tools produce a warning without making its cause clear. A high score can come from one strongly unusual feature, from several weak signals together, or from a rule that recognises a familiar process name. These cases mean very different things, yet they often look the same on the screen. The person reviewing the warning needs to know what was observed, which detector reacted and what information is still missing.

The problem addressed in this project is to turn regular endpoint observations into a monitoring decision that a person can understand and check later. The system must read activity at a fixed interval, learn a reference for normal behaviour, apply more than one detection method and keep enough context to explain each decision after the live values have changed.

Two further issues are part of the problem. The first is collection quality. If a process starts and exits between two readings, the collector never sees it, and a stale value on the dashboard can be mistaken for a fresh one. The collector status, the time of the latest reading and the real collection interval therefore have to be shown next to the risk. The second is the meaning of an ATT&CK label. A process name can suggest a technique without proving that the technique was used, so the system must be able to leave an observation unassigned when the evidence is too thin.

1.3 OBJECTIVES

The main objective is to build an endpoint monitoring application that shows how each of its risk decisions is produced. The prototype brings collection, a behavioural baseline, model inference, rules and alert review into one local workflow. Each objective in Table 1.1 is written as something that can be observed in the running system.

The last objective is reproducibility. A later evaluation should be able to identify the collector, settings, baseline and model used for a run, so that a disagreement between

detectors can be investigated and a revised configuration can be compared with the original one.

Table 1.1 Project Objectives

OBJECTIVE	EXPECTED OUTPUT
Monitor endpoint activity	Timed process, resource and file observations every 1.5 seconds
Learn a behavioural reference	A saved baseline and a deviation score for each new reading
Combine several detectors	One weighted score with a severity level
Explain each decision	Detector contributions and the features that changed most
Support alert review	Search, filters and evidence stored with each alert
Support later evaluation	Telemetry CSV and exported incident records

1.4 SCOPE

The present implementation is a local prototype made of a Python monitoring agent and a Flask dashboard. It observes processes, resource use and changes inside one configured folder. Each observation becomes a ten-feature input that is passed to the digital twin, the Random Forest and the rule engine. The user can check the collector and baseline state, pause or resume monitoring, reset the baseline, study the detector scores and export incidents.

Some areas are deliberately left out of Phase I. The prototype does not claim host isolation, memory scanning, sandbox detonation or correlation across many machines. Sysmon or ETW ingestion and a database for long-term incident history are planned for Phase II. The classifier uses synthetic training data, which is enough to test loading, inference, scoring and display, but cannot show how well it would recognise real malware. Chapter 4 therefore reports what is visible in the running system and lists what still has to be measured.

1.5 SECURITY REQUIREMENTS

A monitoring tool handles information that reveals how a computer is used, so process names, paths and exported incidents should be visible only to authorised reviewers. Integrity matters just as much: a changed model file, feature order or weight can alter every

decision without any visible change on the dashboard. For availability, a failed reading must never turn into a trusted zero.

For the lab demonstration the dashboard is reachable only through the loopback address. Network exposure would need authentication, encryption and input validation, which are listed as deployment requirements and are not claimed as implemented.

Non-Functional Requirements

Explainability is the first non-functional requirement: every displayed risk must be traceable to the detector scores and feature values that produced it. Reliability comes next, since a process that exits during a read, a corrupt model file or a reset in the middle of a cycle must not crash the agent or leave a half-updated result.

The agent runs on the machine it watches, so the 1.5-second cycle should not add a noticeable load during ordinary use. The dashboard listens only on the loopback address, model and baseline files are writable only by the operator, and collection, detection, scoring and presentation are kept in separate modules to ease maintenance.

1.6 SDG ALIGNMENT

The project has a modest, educational link to the Sustainable Development Goals defined by the United Nations [2], [3]. Under Goal 4 it builds practical skills in secure software, data handling and the careful reading of model output. Under Goal 9 it is a small piece of applied research on monitoring methods that could, after proper validation, make computing infrastructure more dependable. No measured change in energy use, employment or infrastructure resilience is claimed.

Table 1.2 SDG Alignment

SDG AND TARGET	PROJECT ALIGNMENT
Goal 4 Quality Education, Target 4.4	Practical skills in secure software, endpoint monitoring, testing and interpretation of results
Goal 9 Industry, Innovation and Infrastructure, Target 9.5	Applied research on a behavioural monitoring prototype with a plan for repeatable evaluation

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Endpoint monitoring tools rely on four broad ideas: signatures and rules, anomaly detection, supervised learning, and the analysis of relationships between events. Signatures and rules match patterns that are already known. Anomaly detection, which Chandola et al. [4] survey across many application areas, flags whatever differs from a learned model of normal behaviour. Supervised learning trains a classifier on labelled examples, while provenance analysis links events such as process creation and file writes into a graph. Each approach answers a different question and depends heavily on the data it receives. Table 2.1 summarises the useful signal and the main limitation of each.

Table 2.1 Detection Approaches Considered

APPROACH	USEFUL SIGNAL	MAIN LIMITATION
Signatures and rules	Known patterns or explicit conditions	Coverage depends on the rule and on the evidence collected
Anomaly detection	Difference from learned behaviour	Normal changes in workload can also look unusual
Supervised learning	Patterns present in labelled data	Unseen workloads may differ from the training data
Provenance analysis	Relationships between events	Needs richer collection and more storage

A typical lightweight monitor, such as a task manager with threshold alerts or a single signature scanner, follows a simple chain: take a snapshot, run one detector and show a label. The label rarely says which value caused it, older readings are overwritten, and the analyst has to rebuild the context by hand. Commercial endpoint detection and response (EDR) platforms are far richer, but they are costly and closed. Karantzas and Patsakis [5] tested a number of commercial EDR products against common attack vectors used in advanced persistent threats and found that several products failed to detect or fully report some of the attacks. Knowing exactly what a tool did and did not observe therefore matters as much as the alert itself.

2.1.1 ALERT QUALITY AND ANALYST WORKLOAD

Alahmadi et al. [1] interviewed analysts working in security operations centres about how they handle alarms. The analysts described very high volumes of false positives and said that alerts often arrive without the context needed to judge them, so even a correct alert can take a long time to confirm. Hassan et al. [6] address the same alert fatigue from the system side. Their approach builds a provenance graph around each alert and scores it by how often similar event paths have appeared in the past, so that common, harmless activity is pushed down the queue and rare chains of events rise to the top.

For Sentinel Twin the main lesson is to keep the explanation next to the alert. A reviewer should not have to rebuild an old decision from whatever the dashboard happens to show now, so every alert stores its own scores, reasons and evidence at the moment it is raised.

2.1.2 PROVENANCE AND EVENT CORRELATION

Hossain et al. [7] showed that attack scenarios can be reconstructed in real time from ordinary operating system audit logs. Their method attaches trust and confidentiality tags to data and follows the dependencies between processes and files. Milajerdi et al. [8] went a step further and matched suspicious information flows to the stages of an advanced persistent threat campaign, producing a compact, high-level graph of the whole attack. Hassan, Bates and Marino [9] applied a similar provenance approach to the alerts produced by existing EDR tools. In an industrial study, Dong et al. [10] reported that runtime overhead, storage cost and the effort needed to interpret large graphs still slow down the adoption of provenance-based tools in practice.

Sentinel Twin does not build provenance graphs yet. It works with aggregate features and selected rule evidence, so a rise in process count can justify a closer look but cannot explain the chain of events behind it. Recording parent and child process links, executable paths and event times is a natural next step, as long as the extra collection and storage cost is measured.

2.1.3 MITRE ATT&CK IN DETECTION ENGINEERING

Strom et al. [11] describe the design of MITRE ATT&CK, a public knowledge base that organises adversary behaviour into tactics and techniques, and the MITRE Corporation [12] maintains the current enterprise matrix. Al-Sada et al. [13] surveyed the research built on ATT&CK and found it used for threat intelligence, detection engineering and the evaluation of defences alike. Virkud et al. [14] studied how real EDR rule sets are labelled with ATT&CK techniques and showed that coverage across techniques is uneven. Rabobank CDC [15] publish an open framework for recording which data sources a team actually collects, so that claims about technique coverage rest on real visibility rather than on rule names.

Sentinel Twin takes a cautious view of technique labels. A reviewed rule can suggest a technique, but a match on a process name alone stays an unverified hypothesis and the alert lists the evidence that is still missing. Unknown rules and general bursts of activity are left unassigned.

2.1.4 MACHINE LEARNING AND EVALUATION BIAS

Sommer and Paxson [16] argued that machine learning is much harder to apply to intrusion detection than to most classification problems, because attacks are rare, normal activity is highly varied and mistakes are expensive. Arp et al. [17] later listed ten common pitfalls in security machine learning, including sampling bias, data snooping, inaccurate labels and evaluation carried out only in the lab. Pendlebury et al. [18] showed that malware classifiers look far stronger than they really are when training and test samples are not separated in time and space. Kaya et al. [19] reached a similar conclusion for behavioural malware detection: models trained on sandbox traces lost much of their accuracy when they were tested on data collected from real endpoints.

These findings shape how the Random Forest results are reported in this project. Synthetic data is useful for checking the workflow and the model interface, but it cannot stand in for the variety of software and users found on real machines. The same caution applies to the twin, since a new but harmless workload can look just as unusual as an attack.

2.1.5 DIGITAL TWINS AND BEHAVIOURAL BASELINES

Tao et al. [20] describe a digital twin as a virtual model that mirrors a physical system and is kept up to date with data from it, so the system can be monitored and analysed without being disturbed. Holmes et al. [21] point out that twins are useful for security but also introduce new risks, because the data and models behind a twin become attractive targets in their own right. Much earlier, Forrest et al. [22] showed that a compact model of normal behaviour, built from short sequences of system calls, was enough to spot unusual behaviour in privileged Unix programs.

Sentinel Twin applies this idea to a single endpoint. Its twin is a reference built from endpoint features, with an anomaly model and per-feature deviations on top. The quality of the twin depends on what it saw during calibration, and if suspicious activity is absorbed into the baseline without review, later attacks may look less unusual. For this reason relearning is always a deliberate user action.

2.2 RESEARCH GAPS

The review points to three practical gaps for a student endpoint prototype. A detection score is often easier to produce than a clear explanation of it. Collection quality and detection quality are often reported together even though they measure different parts of the system. And technique labels frequently look more certain than the evidence behind them. Table 2.2 links each gap to the response in this project and the work that remains.

Table 2.2 Research Gaps and Project Response

GAP	PROJECT RESPONSE	REMAINING WORK
Hard-to-explain scores	Show component scores and feature reasons	Measure how useful the explanations are
Uncertain technique labels	Keep hypotheses and list missing evidence	Review against labelled event records
Missed short-lived activity	Show collector health and timing	Compare with event-based ground truth
Synthetic model evaluation	State the dataset limitation on screen	Evaluate on independent endpoint data

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system builds three detectors around one shared observation. Collection and feature preparation happen once per cycle, and the same ten-feature vector is passed to the twin, the classifier and the rule engine. A scoring stage combines their outputs, assigns a severity and stores the reasons. ATT&CK mapping uses rule evidence only, so a high score never becomes a technique label by itself. Figure 2.1 contrasts this flow with conventional alerting.

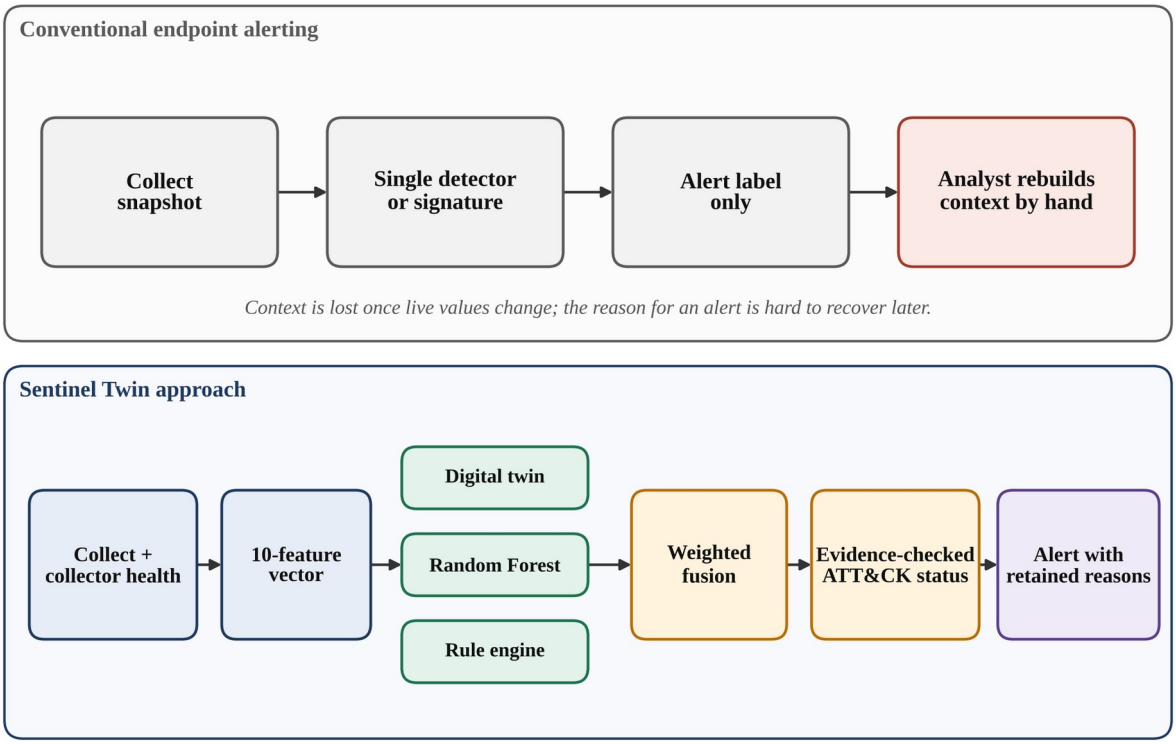


Figure 2.1 Conventional Alerting vs Proposed System

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The functional requirements describe what the system must do. They were collected from the problem statement, the objectives and the behaviour expected during a lab demonstration. Table 2.3 lists each requirement with the module that satisfies it.

Table 2.3 Functional Requirements

ID	REQUIREMENT	MODULE
FR1	Collect process, resource and file observations at a fixed interval	Process and file monitors
FR2	Report collector identity, freshness and read failures	Collector health
FR3	Build a ten-feature vector in a fixed order	Feature builder
FR4	Learn, save and restore a behavioural baseline	Digital twin
FR5	Produce twin, classifier and rule scores for each reading	Detection modules
FR6	Combine the scores and assign a severity	Risk scorer
FR7	Create an alert with reasons for Medium or higher risk	Decision audit
FR8	Link reviewed rules to ATT&CK and record missing evidence	ATT&CK mapper
FR9	Allow pause, resume, baseline reset, filtering and export	Dashboard and agent

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

The main advantage is that several signals can be compared in one place. If the twin is high while the rules are silent, that disagreement stays visible instead of being hidden inside one number. A second advantage is that an older alert keeps its original explanation. Table 2.4 compares the two approaches feature by feature. These are design and usability advantages. Lower false-positive rates or better malware recall still have to be shown by experiment.

Table 2.4 Comparison of Existing and Proposed System

FEATURE	EXISTING MONITORS	SENTINEL TWIN
Reference for normal	Fixed thresholds	Baseline learned on the same machine
Detectors	Usually one	Twin, Random Forest and rules
Explanation	Label only	Component scores and feature reasons
ATT&CK labels	Often implied by rule name	Evidence-checked, may stay unassigned
Collector health	Rarely shown	Interval, freshness and collector shown
Alert history	Overwritten by live values	Alert keeps its own evidence; JSON export

Feasibility Study

A feasibility study was carried out before development to check that the project could be completed with the available time, tools and hardware. Table 2.5 summarises the result.

Table 2.5 Feasibility Study

DIMENSION	ASSESSMENT
Technical	Feasible. Python, psutil, scikit-learn and Flask are mature and well documented, and Isolation Forest and Random Forest are standard algorithms.
Economic	Feasible. All libraries are open source and the system runs on an ordinary student laptop.
Operational	Feasible for a single machine. The dashboard needs no installation beyond a browser.
Security	Feasible with loopback binding, file permissions and careful handling of exported data.
Schedule	The modular design allowed collection, detection and dashboard work to proceed in parallel.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

Sentinel Twin follows a layered architecture, shown in Figure 3.1. Data moves upward from the endpoint through collection, processing, detection and decision to the presentation layer. A local storage column on the right holds the files that outlive one cycle: the model store, the saved baseline, the telemetry CSV, the session alert buffer and the incident exports.

The layers are logical boundaries. The monitoring agent hosts the collection, processing, detection and decision layers in one process, and the Flask server hosts the presentation layer. Keeping the boundaries clear allows one part, such as the collector, to be replaced without changing the meaning of the risk score.

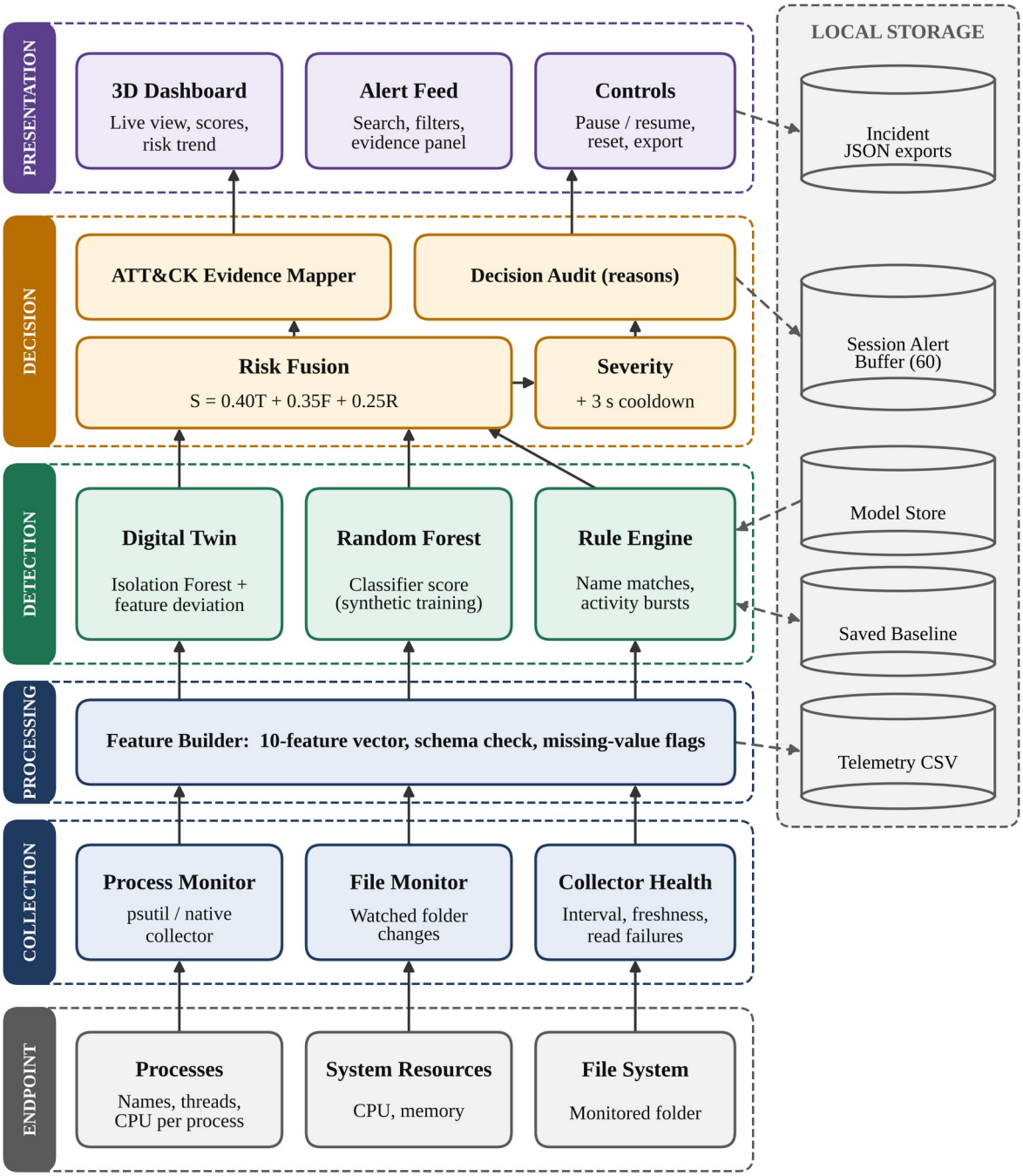


Figure 3.1 System Architecture

Technology Stack

The system is written in Python for the agent and models, with HTML, CSS and JavaScript for the dashboard. Table 3.1 lists the main technologies and the reason each was chosen.

Table 3.1 Technology Stack

AREA	TECHNOLOGY	REASON FOR CHOICE
Collection	psutil, Windows native API	Cross-platform process and resource data
Machine learning	scikit-learn	Isolation Forest and Random Forest in one library
Data handling	NumPy	Feature vectors and numeric operations
Web server	Flask	Small, easy to bind to loopback
Dashboard	HTML, CSS, JavaScript, WebGL	Interactive 3D view and charts

Hardware and Software Requirements

The prototype runs on a single Windows computer with a Python environment and a modern browser. The figures in Table 3.2 are the configuration used for pilot testing. They are suggestions and were not measured as minimum requirements, since overhead depends on the number of processes, the sampling interval, file activity and the 3D rendering load in the browser.

Table 3.2 Hardware and Software Requirements

ITEM	REQUIREMENT
Processor	Multi-core CPU (Intel Core i5 or equivalent suggested)
Memory	8 GB RAM suggested
Storage	At least 1 GB free for models, telemetry and exports
Operating system	Windows 10 or 11 for the native collector
Language	Python 3 with the project's declared dependencies
Main libraries	Flask, scikit-learn, psutil, NumPy
Browser	Chrome, Edge or Firefox with WebGL support

3.2 MODULE DESIGN

The workflow is split into modules so that collection, model behaviour and interface behaviour can be understood and tested separately. Each module has a clear output and a clear failure condition. A collector failure, for example, must never be reported as a valid low-risk reading just because no values reached the detector. Table 3.3 lists the modules.

Table 3.3 Modules and Responsibilities

MODULE	RESPONSIBILITY
Monitoring agent	Coordinates collection, analysis and state changes
Process and file monitors	Read activity and report collection conditions
Feature builder	Builds and validates the ten-feature vector
Digital twin	Compares each observation with the learned reference
Random Forest	Produces a score from the trained classifier
Rule engine	Checks selected process names and bursts
Decision and mapping	Combines scores, gates alerts and records evidence
Dashboard and exports	Presents results and preserves the investigation record

3.2.1 COLLECTION MODULE

The collection module reads process information and resource measurements and records changes in the configured folder. Each cycle is treated as an observation window, not a complete history of the host. A process that exits while being read is a normal condition; the collector logs the failed read and keeps the rest of the cycle usable.

3.2.2 FEATURE PREPARATION MODULE

This module converts the raw readings into the vector used by all three detectors. Each feature must keep the same position and unit during training, calibration and inference, and missing or non-finite values are flagged rather than replaced with normal-looking numbers.

3.2.3 DIGITAL TWIN MODULE

The twin keeps a learned reference for the machine and compares every new observation with it. Initial calibration uses twelve observations, and a valid saved baseline is

restored at start-up. The baseline is tied to the feature schema, so a schema change requires a new calibration.

3.2.4 CLASSIFIER AND RULE MODULES

The classifier gives a learned score based on its synthetic training data, while the rule engine checks explicit conditions such as a known tool name or a burst of file changes. Each returns its score together with the information needed to explain it.

3.2.5 DECISION AND MAPPING MODULE

The decision module fuses the three scores, assigns the severity band, applies the alert cooldown and, for an alert, checks the ATT&CK evidence. The final record holds the timestamp, scores, reasons and mapping status for that single cycle.

3.2.6 DASHBOARD MODULE

The Flask dashboard reads prepared decisions from the agent and never computes scores itself. Its control buttons send pause, resume, reset and export requests back to the agent.

3.3 UML DIAGRAMS

3.3.1 USE CASE DIAGRAM

The main actor is the analyst who runs the dashboard on the local machine. A secondary actor, the Windows endpoint, supplies the process and resource data. Figure 3.2 shows the use cases started by the analyst on the left and the use cases carried out by the system on the right. Include relations mark steps that always happen as part of a user action. The extend relation from baseline calibration to start monitoring applies only when no valid saved baseline exists. Raising an alert extends risk scoring because it happens only when the score reaches the Medium band.

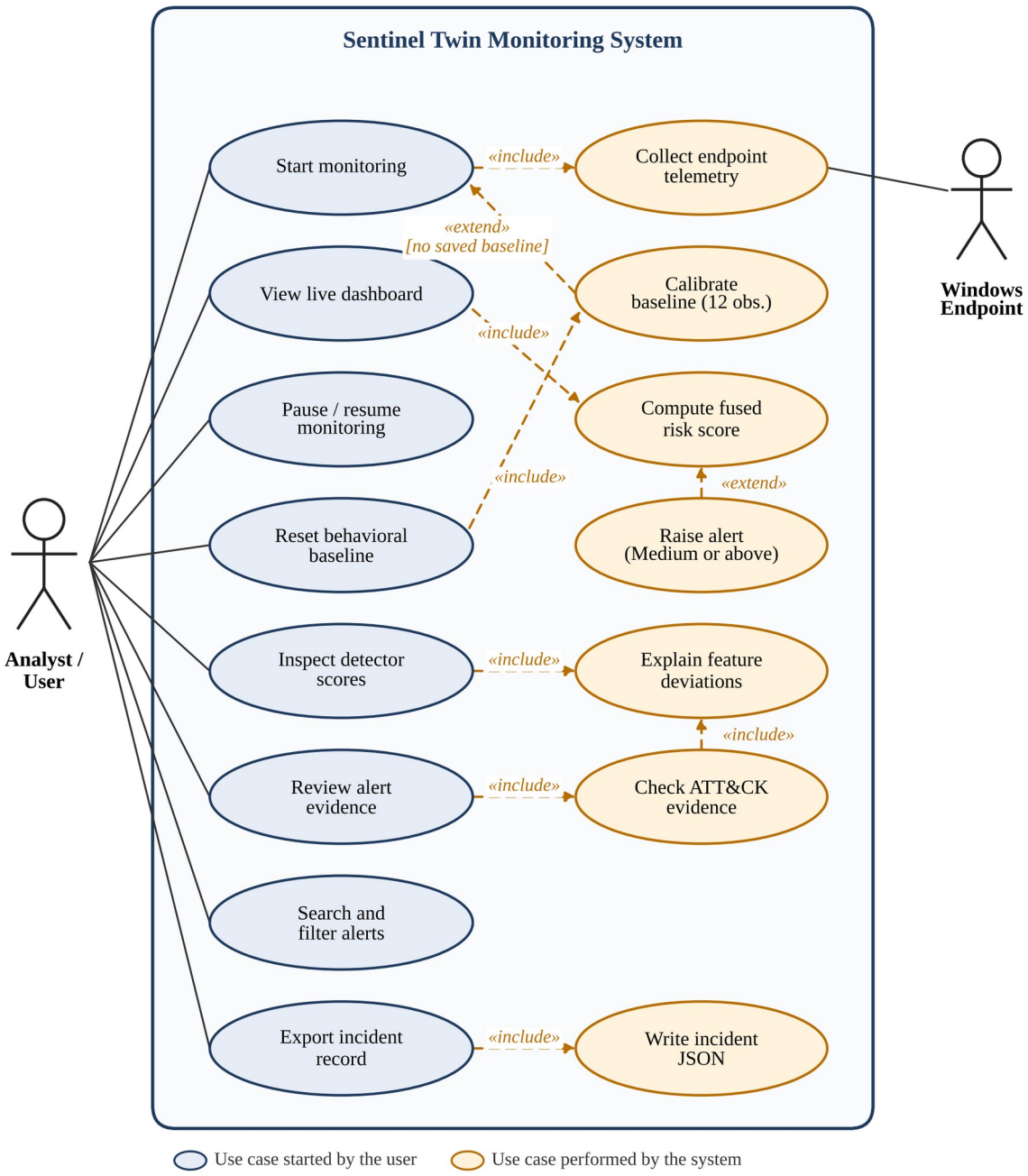


Figure 3.2 Use Case Diagram of Sentinel Twin

Table 3.4 describes the main use cases in more detail. Automatic host containment is not a use case. The analyst stays responsible for any action taken outside the application, because a model score or a name-only technique match is not enough reason to stop a process that may be legitimate.

Table 3.4 Use Case Descriptions

USE CASE	DESCRIPTION
Start monitoring	Agent loads models and baseline, then begins collection. If no valid baseline exists, twelve readings are used for calibration first.
View live dashboard	Shows current readings, collector status, component scores, fused risk and the risk trend.
Pause / resume	Stops new collection without clearing existing alerts. The dashboard shows the Paused state clearly.
Reset baseline	Discards the current reference and starts a new calibration cycle under user control.
Inspect detector scores	Opens the score panel with feature deviations and the classifier and rule results.
Review alert evidence	Shows the stored reasons, ATT&CK status and missing evidence for one alert.
Search and filter alerts	Filters the session alert feed by time, severity or text.
Export incident	Writes the selected alert and its evidence to a JSON file for later review.

3.3.2 WORKFLOW DIAGRAM

Figure 3.3 shows the complete workflow of the system from start-up to the repeating monitoring loop. At start-up the agent loads its configuration and model files and checks that they match the expected feature schema (steps 1 and 2). If a model cannot be loaded, the recovery path reports the problem and reloads a trusted copy instead of continuing with a broken model. The agent then looks for a valid saved baseline (step 3). When none is found, it collects twelve observations and fits the Isolation Forest before any score is shown.

The loop begins at step 4. If the user has paused monitoring, the agent holds its last decision and waits. Otherwise it collects a new observation and builds the feature vector (steps 5 and 6). An incomplete reading is marked as such, the collector health is updated, and scoring is skipped for that cycle so that missing values never pass as a normal reading (step 7). A complete vector goes to the three detectors and then to the fusion stage (steps 8 and 9). If the result is Medium or above and the three-second cooldown has passed, an alert is created with its reasons and ATT&CK status (step 10). In every case the dashboard and telemetry are updated (step 11) and the loop starts again after the 1.5-second interval.

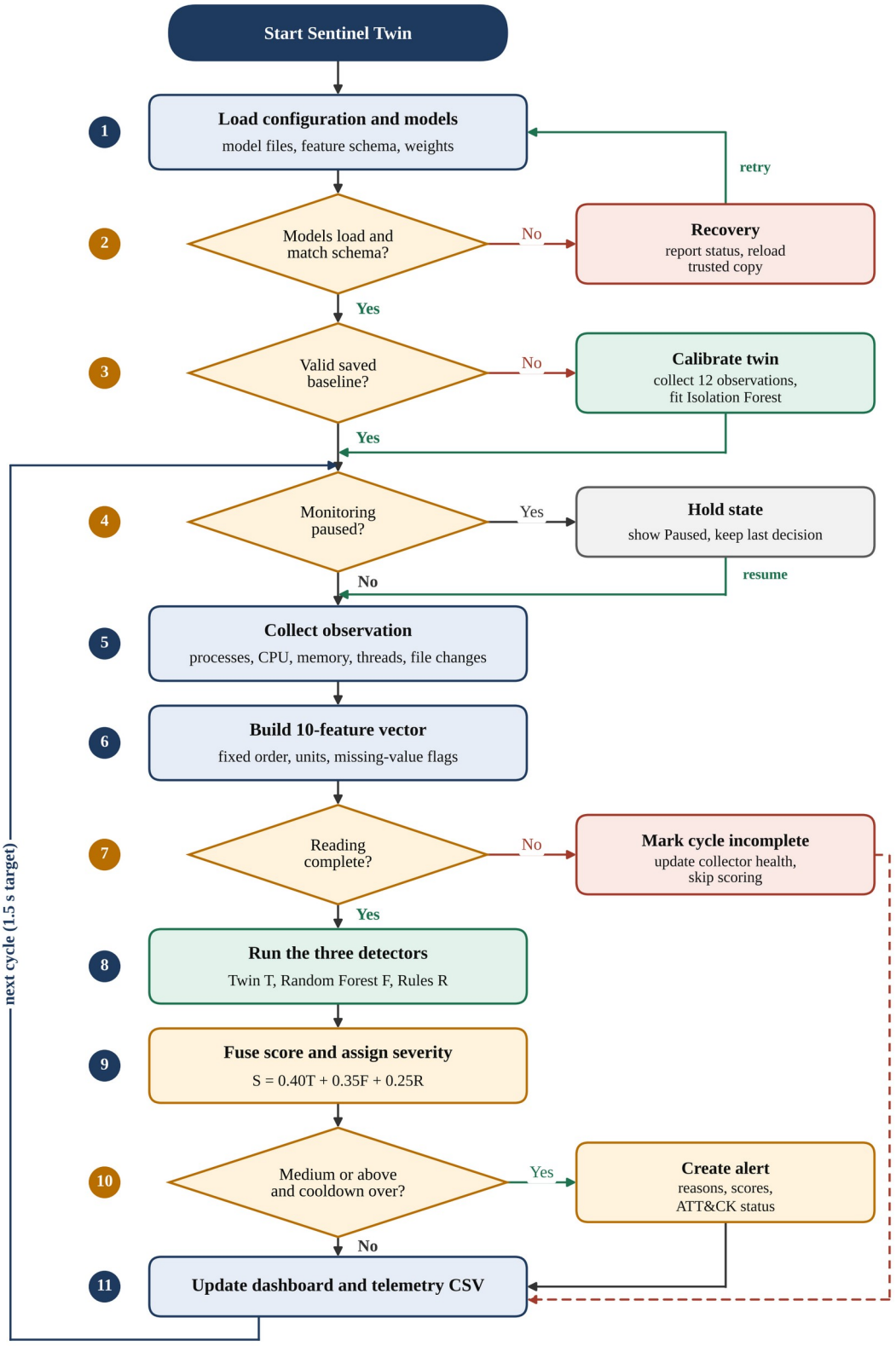


Figure 3.3 System Workflow

3.3.3 DATA FLOW DIAGRAMS

Level 0 Data Flow Diagram

The context diagram in Figure 3.4 treats the whole system as one process. Four external entities interact with it. The Windows endpoint supplies process and resource readings, the watched folder supplies file change events, the analyst sends control commands and receives the live view, and the analyst's record files receive the telemetry rows and incident exports.

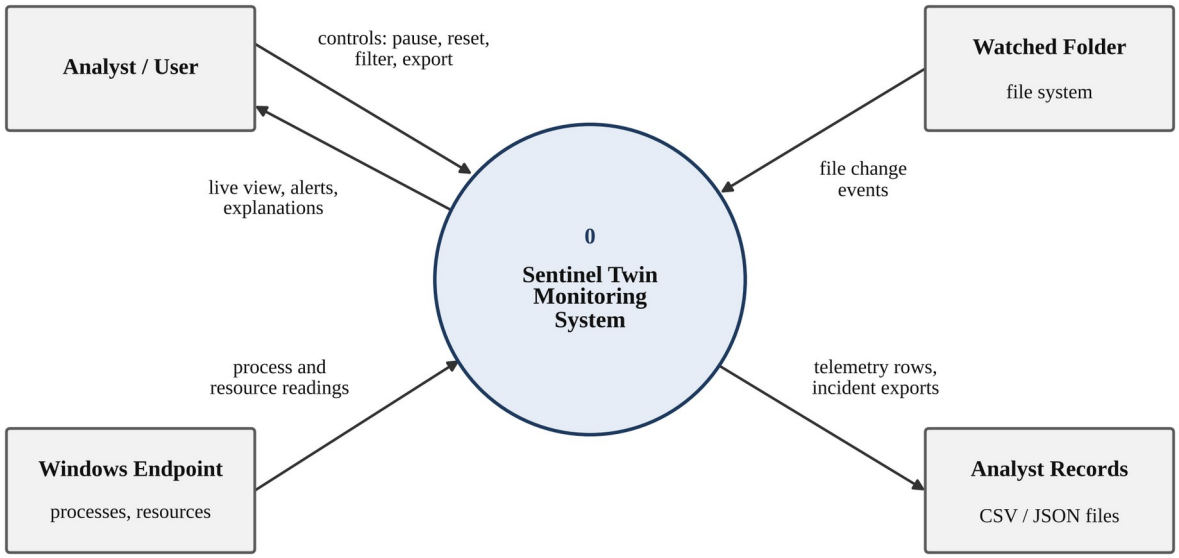


Figure 3.4 Level 0 Data Flow Diagram

Level 1 Data Flow Diagram

Figure 3.5 breaks the system into six processes and four data stores. Process 1.0 collects raw readings, 2.0 turns them into a feature vector, and 3.0 runs the detectors using the saved baseline (D1) and the model files (D2). Process 4.0 fuses the three scores, assigns a severity and appends a row to the telemetry CSV (D3). Process 5.0 checks ATT&CK evidence and writes alerts to the alert buffer and incident files (D4). Process 6.0 presents the results to the analyst and handles control commands.

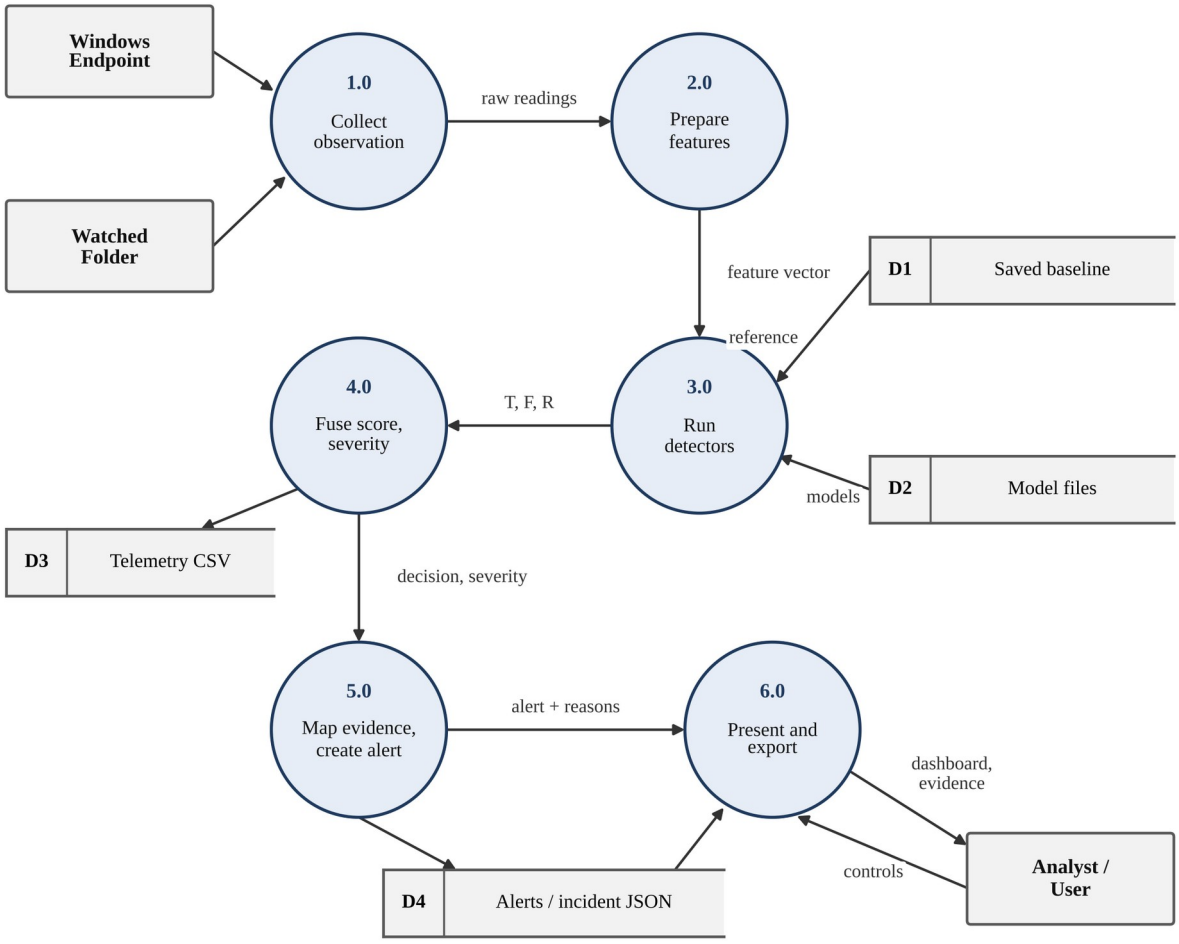


Figure 3.5 Level 1 Data Flow Diagram

3.3.4 SEQUENCE DIAGRAM

Figure 3.6 shows the order of messages in one monitoring session. The analyst opens the dashboard, which asks the agent for its status. While the agent is in the Monitoring state it repeats the loop every 1.5 seconds: it asks the collectors for an observation, asks the feature builder for the vector, sends the vector to the detectors and passes the three scores to the risk scorer. The scorer works out the severity and checks the cooldown. Only when the result is Medium or higher does it check the ATT&CK evidence and return an alert record. The agent then pushes the decision to the dashboard. Later the analyst can open an alert and export it.

The timestamp, feature values, component scores and explanation in each message all belong to the same cycle, so the interface can never show an old score next to new feature values.

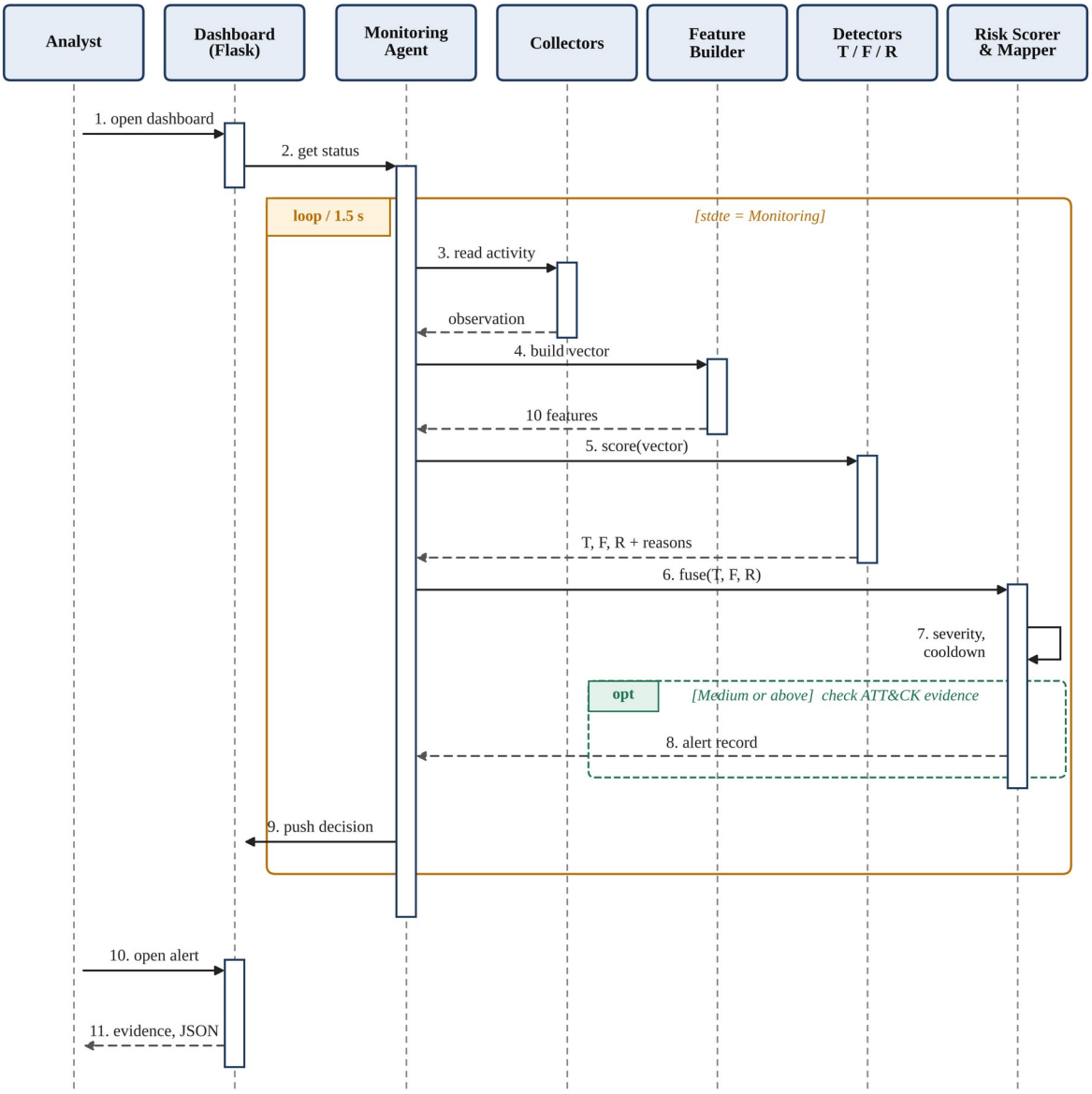


Figure 3.6 Sequence Diagram of a Monitoring Session

3.3.5 STATE DIAGRAM

The agent moves between six states, shown in Figure 3.7 and described in Table 3.5. A score shown during calibration means something different from a score calculated against a finished baseline, and an unchanged dashboard during pause does not mean the endpoint is idle. Showing the state clearly prevents both mistakes. State changes are coordinated with collection so that a reset cannot leave a half-updated baseline attached to a decision that looks complete.

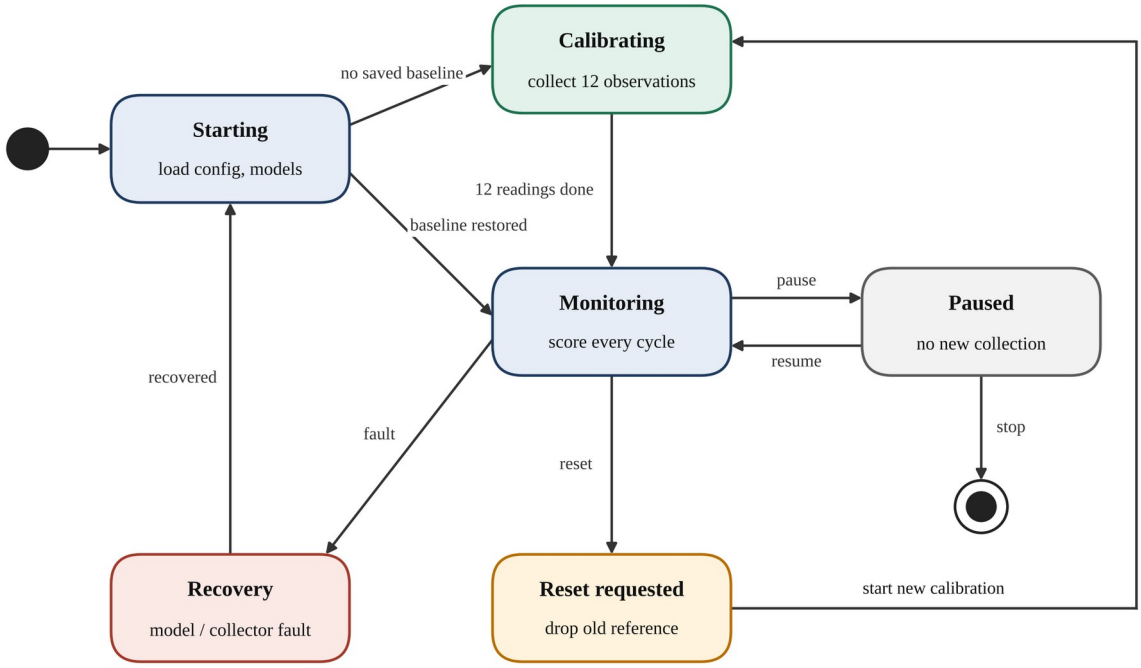


Figure 3.7 State Diagram of the Monitoring Agent

Table 3.5 Monitoring States

STATE	MEANING	NEXT ACTION
Starting	Configuration and models are checked	Restore baseline or calibrate
Calibrating	Reference observations are collected	Enter monitoring after twelve readings
Monitoring	New observations are scored	Continue, pause or reset
Paused	Collection is stopped for now	Resume monitoring or stop
Reset requested	The old reference is being replaced	Start a new calibration
Recovery	A model or collector fault is handled	Report status and return to Starting

3.3.6 CLASS DIAGRAM

Figure 3.8 shows the main data objects and how they relate. An Observation produces one FeatureVector, which is compared with the current Baseline and scored by the twin and classifier models described in ModelInfo. The result is a Decision holding the three component scores, the fused score, the severity and the reasons. A Decision raises at most

one Alert, and an Alert can carry ATT&CK mapping records and be written out as one or more incident exports. The attribute names describe the logical model used in this report and may differ slightly from the names in the source code.

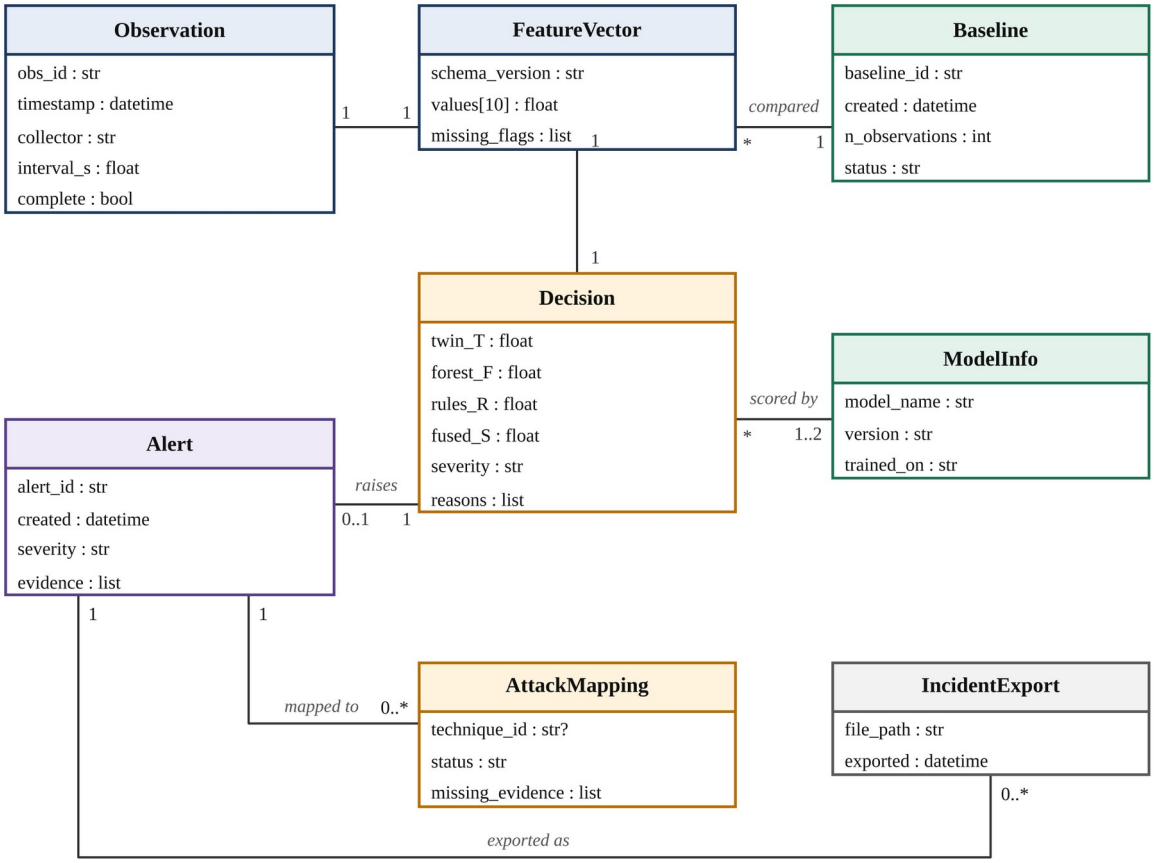


Figure 3.8 Class Diagram of the Data Model

3.3.7 DEPLOYMENT DIAGRAM

Figure 3.9 shows the deployment on one Windows machine. The agent and the Flask server share a Python virtual environment, the browser connects only on 127.0.0.1:5000, and all files stay on the local disk.

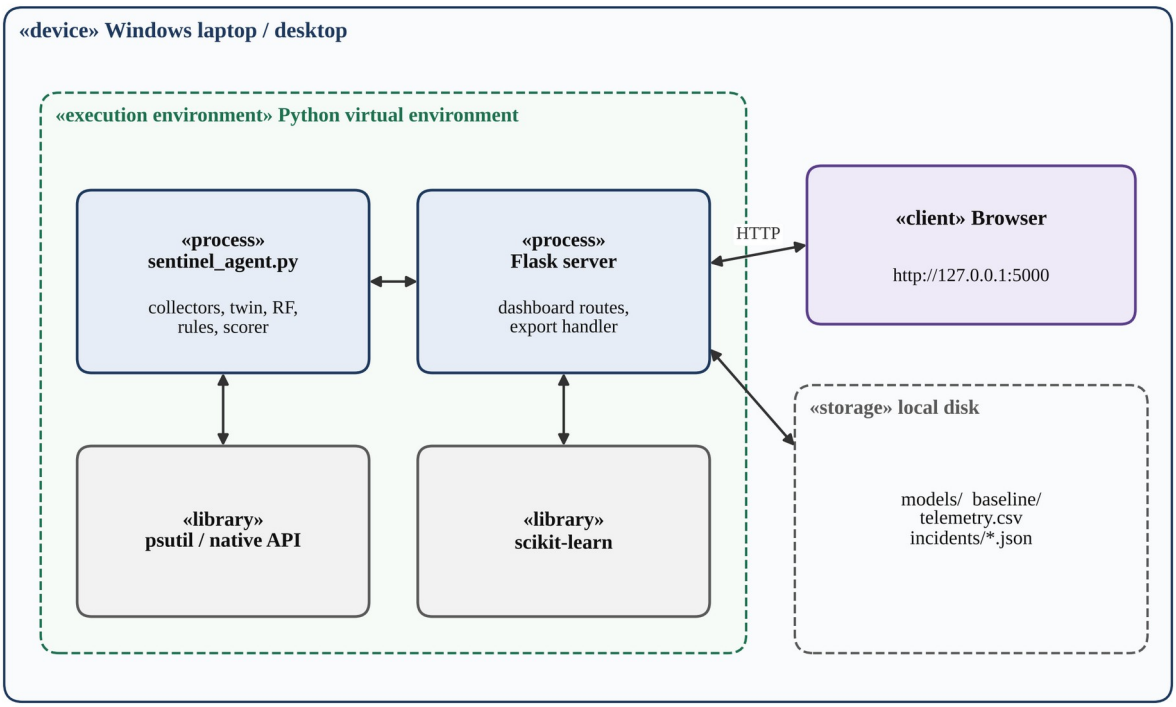


Figure 3.9 Deployment Diagram

3.4 THREAT MODEL AND ATTACK SURFACE

The assets are monitoring continuity, trustworthy observations, the integrity of models and baselines, and the confidentiality of recorded activity. The model assumes that the lab user is authorised to monitor the machine. It does not claim to resist an attacker who already has full administrator control of the operating system and the application files. Figure 3.10 shows the trust boundaries and marks where each threat in Table 3.6 applies. Process names and file paths come from the monitored system and are treated as untrusted data. They are displayed and exported as plain text and are never interpreted as commands or markup.

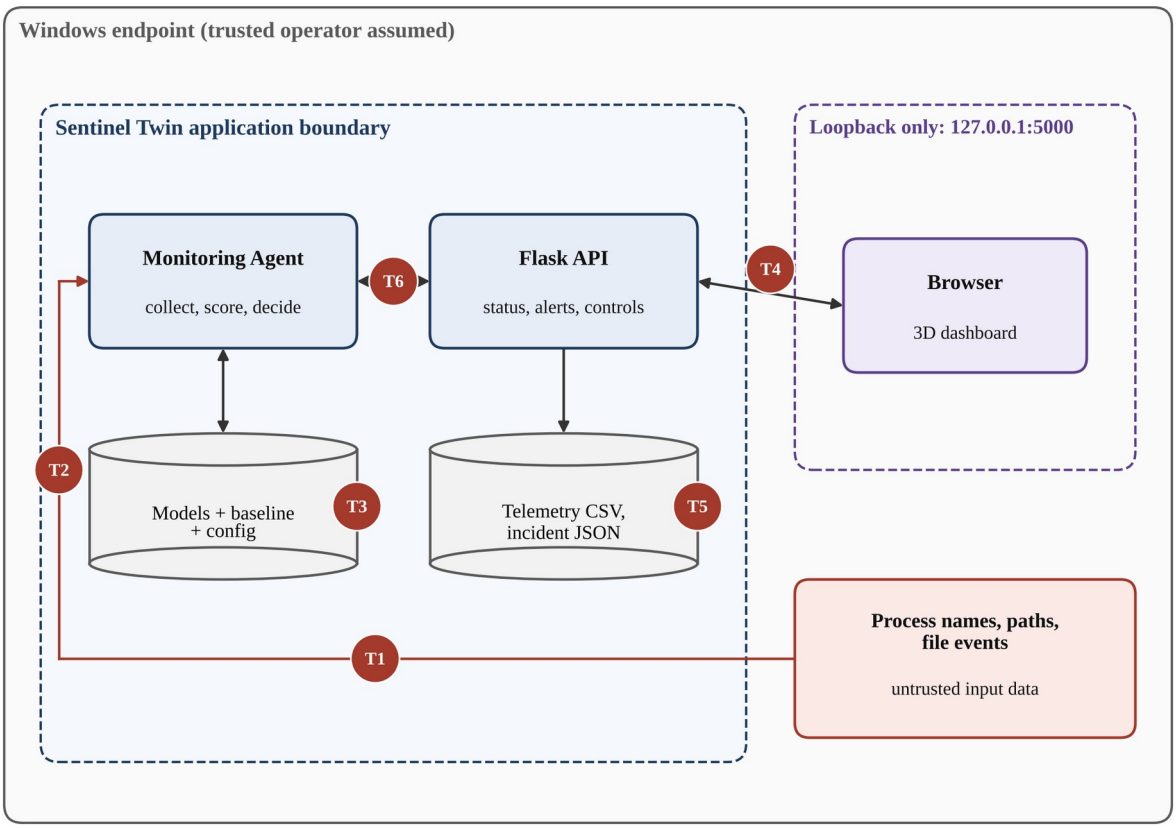


Figure 3.10 Trust Boundaries and Threat Locations

Table 3.6 Threats and Their Impact

ID	THREAT	POSSIBLE EFFECT
T1	Misleading process identity	A renamed tool makes a name match look more certain than the evidence
T2	Missed or delayed collection	The detectors judge an incomplete picture of the endpoint
T3	Model or baseline modification	Scores change without a valid experimental reason
T4	Unsafe dashboard exposure	An unauthorised user reads data or changes monitoring
T5	Sensitive export disclosure	Process names or paths reveal user activity
T6	Uncoordinated state change	A reset or pause leaves inconsistent results

3.5 SECURITY CONTROLS

Table 3.7 lists the controls and separates those present in the prototype from those planned for a hardened release. Treating both groups as implemented would overstate the security of the prototype.

Table 3.7 Security Controls

CONTROL	STATUS	ADDRESSES
Evidence-checked ATT&CK status	Present	T1, overconfident labels
Collector health and freshness display	Present	T2
Corrupt model detection and recovery	Present	T3 (accidental damage)
Loopback-only dashboard	Present	T4
Coordinated pause and reset handling	Present	T6
Model metadata and hash checks	Planned	T3 (deliberate tampering)
Baseline versioning and rollback	Planned	T3, poisoned baseline
Authentication for remote access	Planned	T4
Export review and restricted folder	Operator practice	T5

Privacy of Recorded Data

Telemetry and incident files stay on the local disk and are not uploaded anywhere. Before an export is shared for a report or a demonstration, it should be checked for user names, private folder paths and any other identifying details. Old telemetry should be deleted when it is no longer needed for an experiment. A future database will need the same retention rules, since storing records for longer does not by itself make them safe or tamper-resistant.

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The collector supplies process observations, CPU and memory readings, thread activity, selected name matches and file changes from the watched folder. Process and resource readings are gathered through the native Windows collector and the psutil library developed by Rodola [23]. These readings describe what is visible at the moment the collector runs, so very short-lived processes can be missed between two readings. The target interval is 1.5 seconds. Sysmon, the system monitoring service published by Microsoft [24], is a candidate source for event-based collection in Phase II, but it is not part of the present prototype. Table 4.1 lists the information collected in each cycle and what it can show.

Table 4.1 Collected Information

INFORMATION	WHAT IT CAN SHOW
Process activity	Processes visible during the collection window
Resource use	System CPU, memory, total threads and highest process CPU
File activity	Created, changed and deleted files in the watched folder
Collector status	Active collector, observed interval and freshness of readings

Dashboard and Exports

The single-page dashboard shows the agent state, collector, interval and baseline status at the top, a 3D view with a risk gauge, a trend chart of T, F, R and S, the explanation panel and a searchable alert feed. Telemetry rows go to a CSV file every cycle, and any alert can be exported as JSON.

4.2 DATA PREPROCESSING

Each cycle produces one vector of ten numerical features covering process count, CPU, memory, threads, name matches and file activity. The exact order and units are defined in the project's feature schema, and the same schema is saved with the models and the baseline. Missing readings, unavailable process attributes and non-finite values are flagged. A

flagged cycle is shown as incomplete instead of being converted into a normal-looking reading. Context that is not a model input, such as collection time, collector identity and rule evidence, travels with the vector so that the final decision can be explained.

4.2.1 ALGORITHMS

Digital Twin (Isolation Forest)

The twin uses the Isolation Forest algorithm introduced by Liu et al. [25]. The algorithm builds many random trees, each of which splits the data on a randomly chosen feature at a random value. Unusual readings are separated from the rest after only a few splits, so a short average path length points to an unusual observation. The twin is fitted on the calibration readings and then scores every new vector.

To explain a high score, the twin also measures how far each feature has moved from its baseline, expressed in units of that feature's usual spread. A feature that normally varies very little therefore stands out quickly when it changes, while a feature that is always noisy needs a much larger change before it is reported. If a feature never varied during calibration, a safe fallback is used.

The features that moved furthest are shown in the explanation panel. The raw anomaly score from the library is converted to the 0 to 100 scale used by the fusion stage, and the conversion settings are saved with the model configuration. The main steps of the twin are as follows.

Step 1: Collect twelve calibration readings from a known benign workload. Step 2: Fit the Isolation Forest and store the usual level and spread of each feature. Step 3: For each new reading, compute the anomaly score and convert it to the range 0 to 100. Step 4: Find the features that moved furthest from their usual level and keep them as reasons. Step 5: Return the score and the reasons to the agent.

Random Forest Classifier

Breiman [26] introduced the Random Forest, which combines the votes of many decision trees, each trained on a random sample of the data and of the features. In Sentinel Twin the classifier is built with the scikit-learn library described by Pedregosa et al. [27] and is trained on synthetic benign-like and malicious-like patterns. Its role in Phase I is to provide

a second, learned signal inside the monitoring loop. Its output is shown as a score from 0 to 100 together with a label such as benign-like. Because its probability calibration has not been tested on real data, the score is not presented as the chance of an attack, and the dashboard reminds the user that the training data is synthetic.

Rule Engine

The rule engine checks two kinds of conditions. Identity rules match process names against a short reviewed list of tools often seen in attacks. Burst rules fire when an activity count, such as the number of file changes in the watched folder, rises sharply within one cycle. Each rule that fires adds to the rule score and records the condition and the observed value. A rule match is reported as an observation, for example that a certain name was seen, and not as proof of intent, since names can be copied and bursts also happen during installs and builds.

Risk Fusion and Alert Generation

The fusion stage combines the three component scores with fixed weights. The twin score carries 40 percent of the weight, the Random Forest score 35 percent and the rule score 25 percent. The weighted scores are added to give the fused risk score.

Because the weights add up to one, the fused score stays between 0 and 100. A score below 30 is Low, 30 to below 55 is Medium, 55 to below 80 is High, and 80 or above is Critical. A decision of Medium or above creates an alert, provided at least three seconds have passed since the previous one. This cooldown only limits how often alerts are created and does not pause collection. Figure 4.1 shows how the scores combine for the reading captured in the dashboard screenshot, where the twin gave 80, the Forest 32 and the rules 0, and the fused score of 43.3 fell in the Medium band. The default scoring and monitoring settings are listed in Table 4.2.

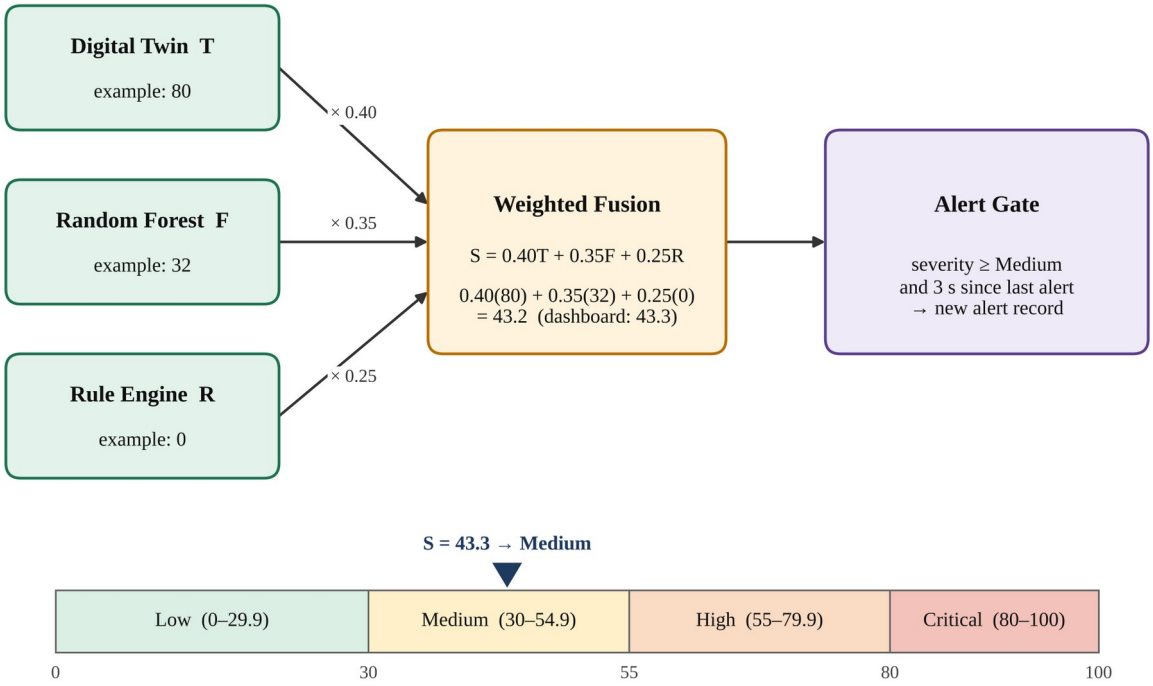


Figure 4.1 Risk Fusion and Severity Bands

Table 4.2 Default Scoring and Monitoring Settings

SETTING	VALUE
Twin / Forest / Rule weights	0.40 / 0.35 / 0.25
Initial calibration	12 observations
Collection target	1.5 seconds
Alert condition	Medium or higher, 3-second cooldown
Retained session alerts	Up to 60
Baseline relearning	Only on user request

ATT&CK Mapping

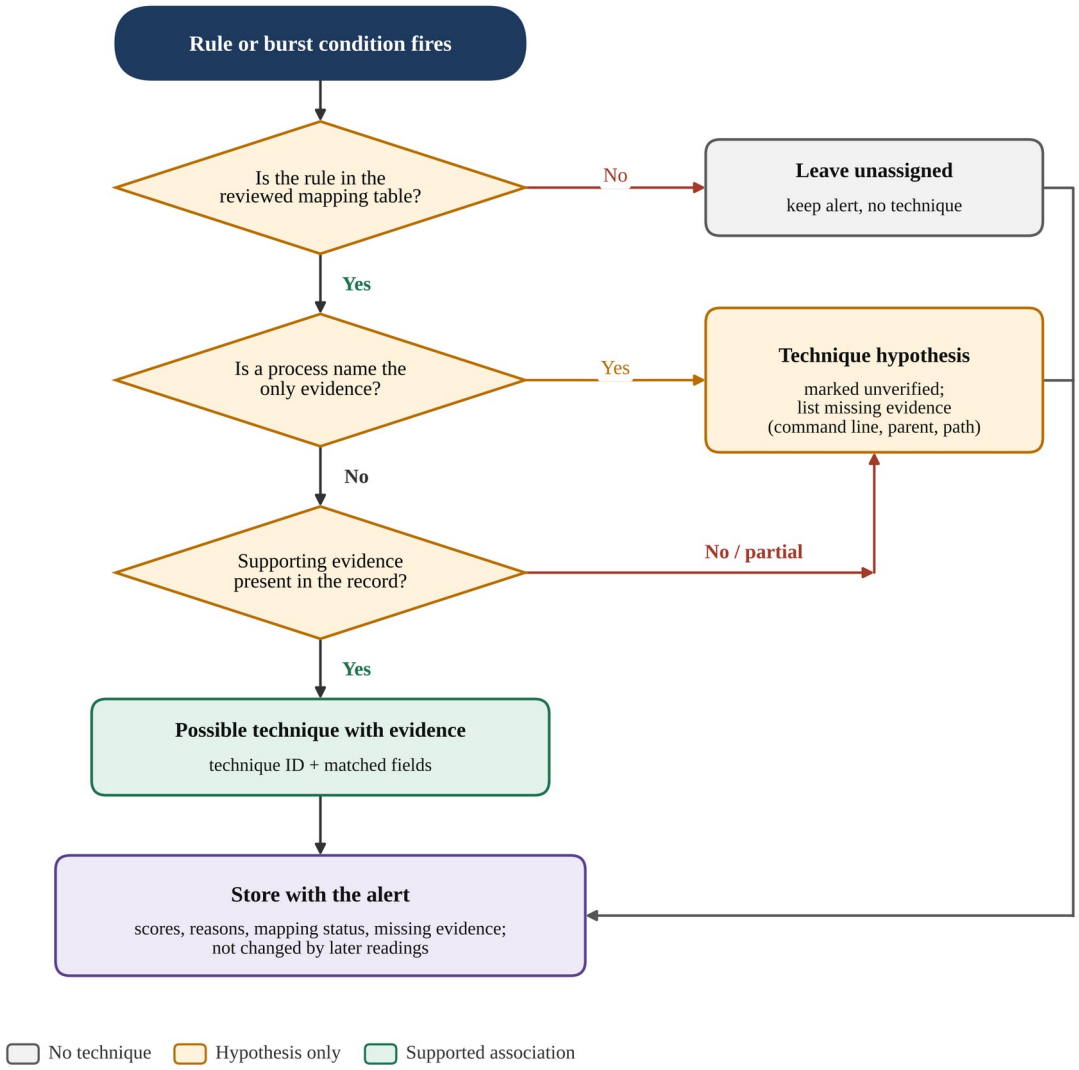


Figure 4.2 Evidence Checks for ATT&CK Mapping

The mapping step follows the checks in Figure 4.2. If the rule that fired is not in the reviewed mapping table, the alert is kept but no technique is assigned. If the only evidence is a process name, the technique is stored as an unverified hypothesis and the alert lists what is missing, such as the command line, parent process or executable path. Only when supporting evidence is present is the technique recorded as a possible association with the matched fields. Aggregate bursts are left unassigned, since an unusual workload can deserve a look without pointing to one technique.

4.2.2 RESULTS AND DISCUSSION

Monitoring Dashboard Overview

Figure 4.3 shows the dashboard during a live session. At the moment of capture it reports 339 active processes and 78.4 percent system CPU usage. The native collector is active, the baseline has been restored from disk, and the timing card shows an observed interval of 1.53 seconds against the 1.50-second target. Placing collector health next to the detector view lets the user confirm that fresh readings are arriving before trusting a score. These values describe a single moment and do not measure average overhead or long-run timing. The CPU figure is for the whole endpoint, not for Sentinel Twin itself.

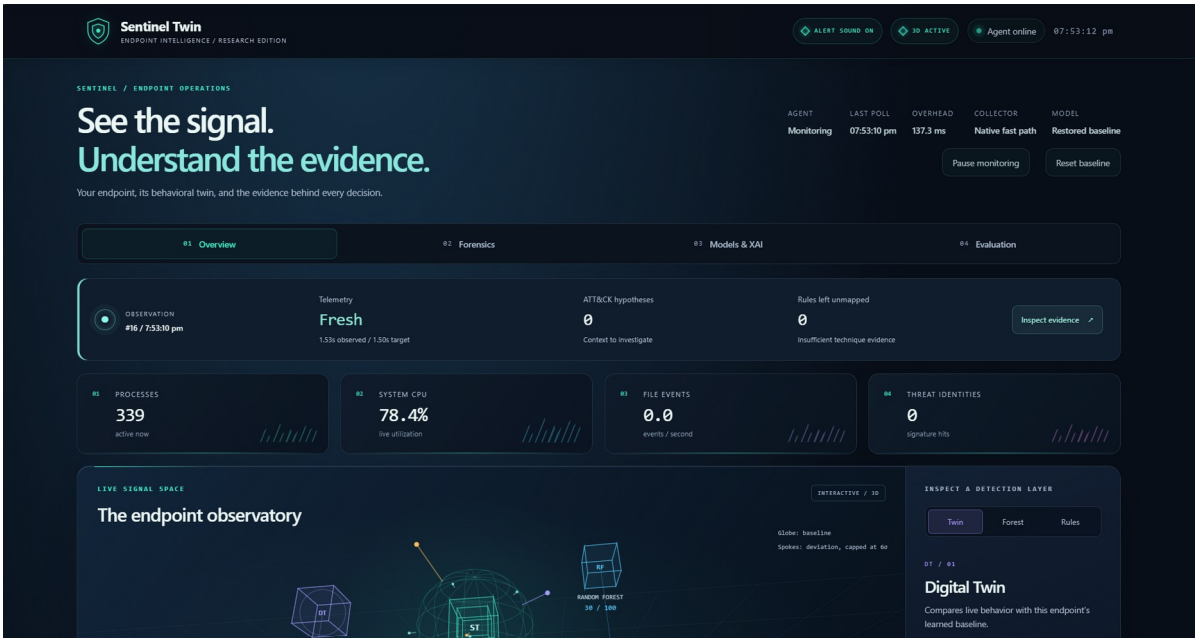


Figure 4.3 Monitoring Dashboard Overview

Detector Scores and Risk Trend

Figure 4.4 shows a twin score of 80, a Forest score of 32 and a rule score of 0. The fused score is 43.3, which places the reading in the Medium band. The twin is clearly the main contributor, while the classifier sees a benign-like pattern and no rule has fired. Because the three scores are shown separately, this disagreement is easy to see. An analyst would start by looking at how the reading differs from the baseline, rather than assuming a rule confirmed an attack. The trend chart adds time context, though detection delay and false-alert rate cannot be measured from a screenshot alone.



Figure 4.4 Detector Scores and Risk Trend

Feature Explanations and Alert Feed

The explanation panel in Figure 4.5 names the features that moved most: total thread count, system CPU and the highest single-process CPU. It also shows the benign-like Forest result and the absence of identity or burst rule matches. The alert feed on the right supports search, time and severity filters, and opens the stored evidence for each alert. The panel shows a holdout accuracy of 100.0 percent and a ROC AUC of 1.000. These numbers come from the synthetic classifier dataset and confirm that the training pipeline works. They are not real-world detection results.

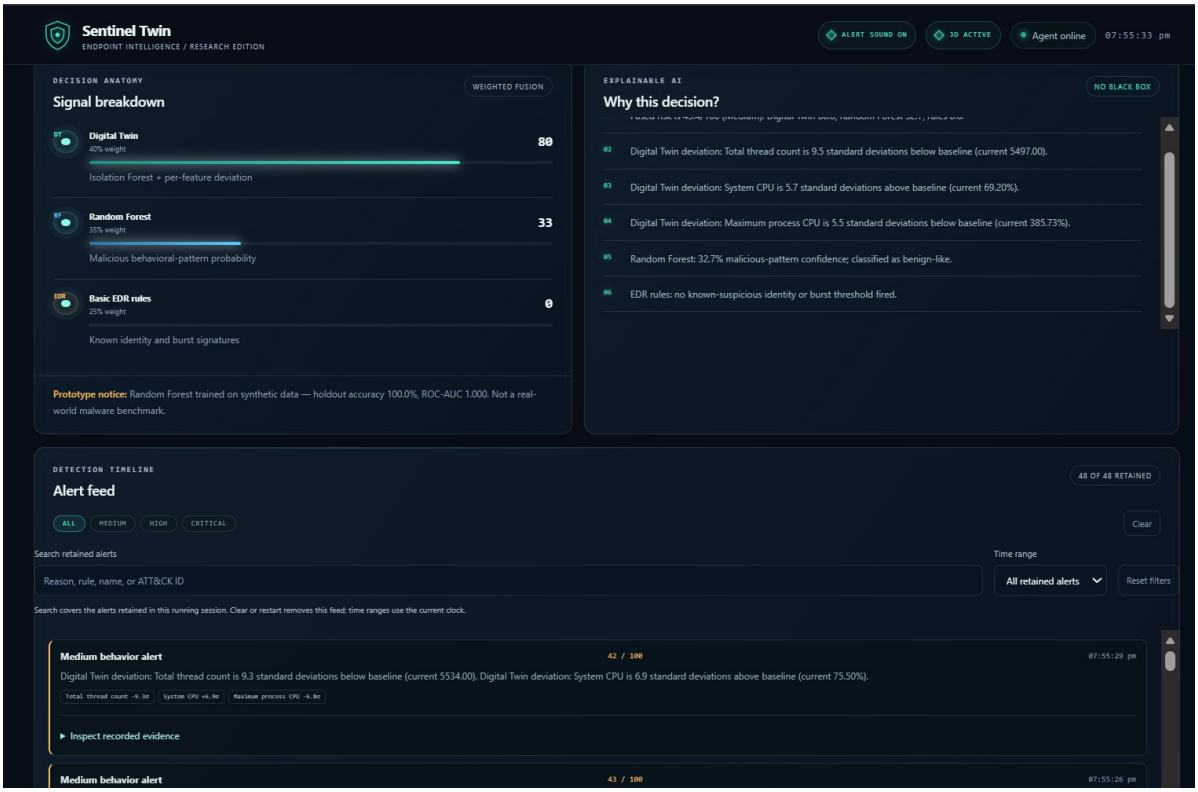


Figure 4.5 Feature Explanations and Alert Feed

Discussion and Limitations

The main limitations are the synthetic training data, the short twelve-reading calibration and the activity that can be missed between polling intervals. Aggregate resource features carry less context than process ancestry, command lines and correlated file or network events. The default weights have not been tuned through an independent comparison. Finally, the sixty-alert session buffer is suitable for a demonstration but is not a replacement for durable incident storage, so important alerts should be exported before the session is cleared.

CHAPTER 5

CONCLUSION

Sentinel Twin provides a local endpoint monitoring workflow built around a behavioural digital twin, a Random Forest classifier and a small set of rules. The system collects observations every 1.5 seconds, prepares a ten-feature input, calculates three component scores and combines them into a severity level. Its dashboard lets the user examine the reasons for each result instead of relying on a warning label alone.

The main contribution of Phase I is the integration of monitoring and explanation. Detector contributions stay visible, feature deviations describe what changed, each alert keeps the context of its original decision, and every ATT&CK association carries an explicit status.

The present results confirm that the monitoring and review workflow operates as designed. They do not establish malware detection accuracy in deployment, because the classifier uses synthetic data, calibration is short and polling can miss activity. Phase II is planned around exactly these points.

Future Work

Phase II will start with richer, timed endpoint evidence. Sysmon or ETW integration can provide process creation events, parent and child links and command lines that periodic snapshots cannot capture reliably, and the gain in visibility will be measured against a reference event record. The second priority is evaluation data covering ordinary browsing, development, updates and file operations as benign workloads, together with isolated, authorised attack simulations such as the Atomic Red Team library maintained by Red Canary [28], each with a clear record of the evidence it should produce.

The third priority is reproducibility and storage. Models and baselines will receive version numbers, decisions will link to their configuration, and a database will keep incident history beyond one session. Automated response will be considered only after detection and evidence quality have been measured. Table 5.1 lists the planned tasks and how each will be judged complete.

Table 5.1 Phase II Work Plan

TASK	COMPLETION EVIDENCE
Event-based collection	Measured recall and timing against reference events
Independent evaluation	Reviewed labels and separate test workloads
Detector comparison	Results for each detector alone and combined
Persistent records	Recoverable incidents linked to configuration
Baseline management	Version history, review and rollback
Response design	Reviewed action policy and recovery procedure

REFERENCES

1. Alahmadi, Bushra A., Louise Axon, and Ivan Martinovic. “99% False Positives: A Qualitative Study of SOC Analysts’ Perspectives on Security Alarms.” *Proceedings of the 31st USENIX Security Symposium*, USENIX Association, 2022.

2. United Nations. “Goal 4: Quality Education.” *Sustainable Development Goals*, sdgs.un.org/goals/goal4. Accessed 1 Oct. 2026.

3. United Nations. “Goal 9: Industry, Innovation and Infrastructure.” *Sustainable Development Goals*, sdgs.un.org/goals/goal9. Accessed 1 Oct. 2026.

4. Chandola, Varun, Arindam Banerjee, and Vipin Kumar. “Anomaly Detection: A Survey.” *ACM Computing Surveys*, vol. 41, no. 3, 2009, pp. 1–58.

5. Karantzas, George, and Constantinos Patsakis. “An Empirical Assessment of Endpoint Detection and Response Systems against Advanced Persistent Threats Attack Vectors.” *Journal of Cybersecurity and Privacy*, vol. 1, no. 3, 2021, pp. 387–421.

6. Hassan, Wajih Ul, et al. “NoDoze: Combatting Threat Alert Fatigue with Automated Provenance Triage.” *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2019.

7. Hossain, Md Nahid, et al. “SLEUTH: Real-Time Attack Scenario Reconstruction from COTS Audit Data.” *Proceedings of the 26th USENIX Security Symposium*, USENIX Association, 2017, pp. 487–504.

8. Milajerdi, Sadegh M., et al. “HOLMES: Real-Time APT Detection through Correlation of Suspicious Information Flows.” *Proceedings of the 2019 IEEE Symposium on Security and Privacy*, IEEE, 2019, pp. 1137–1152.

9. Hassan, Wajih Ul, Adam Bates, and Daniel Marino. “Tactical Provenance Analysis for Endpoint Detection and Response Systems.” *Proceedings of the 2020 IEEE Symposium on Security and Privacy*, IEEE, 2020, pp. 1172–1189.

10. Dong, Feng, et al. “Are We There Yet? An Industrial Viewpoint on Provenance-Based Endpoint Detection and Response Tools.” *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ACM, 2023, pp. 2396–2410.

11. Strom, Blake E., et al. *MITRE ATT&CK: Design and Philosophy*. Technical Report MP180360, The MITRE Corporation, 2018.

12. The MITRE Corporation. “MITRE ATT&CK Enterprise Matrix.” *MITRE ATT&CK*, attack.mitre.org. Accessed 15 Sept. 2026.

13. Al-Sada, Bader, Alireza Sadighian, and Gabriele Oligeri. “MITRE ATT&CK: State of the Art and Way Forward.” *ACM Computing Surveys*, vol. 57, no. 1, 2024, pp. 1–37.

14. Virkud, Apurva, et al. “How Does Endpoint Detection Use the MITRE ATT&CK Framework?” *Proceedings of the 33rd USENIX Security Symposium*, USENIX Association, 2024, pp. 3891–3908.

15. Rabobank CDC. “DeTT&CT: Detect Tactics, Techniques & Combat Threats.” *GitHub*, github.com/rabobank-cdc/DeTTECT. Accessed 15 Sept. 2026.

16. Sommer, Robin, and Vern Paxson. “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection.” *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, IEEE, 2010, pp. 305–316.

17. Arp, Daniel, et al. “Dos and Don’ts of Machine Learning in Computer Security.” *Proceedings of the 31st USENIX Security Symposium*, USENIX Association, 2022, pp. 3971–3988.

18. Pendlebury, Feargus, et al. “TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time.” *Proceedings of the 28th USENIX Security Symposium*, USENIX Association, 2019, pp. 729–746.

19. Kaya, Yigitcan, et al. “ML-Based Behavioral Malware Detection Is Far from a Solved Problem.” *Proceedings of the IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, IEEE, 2025, pp. 921–940.

20. Tao, Fei, et al. “Digital Twin in Industry: State-of-the-Art.” *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, 2019, pp. 2405–2415.

21. Holmes, Duncan, et al. “Digital Twins and Cyber Security – Solution or Challenge?” *Proceedings of the 6th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference*, IEEE, 2021, pp. 1–8.

22. Forrest, Stephanie, et al. “A Sense of Self for Unix Processes.” *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, IEEE, 1996, pp. 120–128.

23. Rodola, Giampaolo. “psutil: Cross-Platform Process and System Monitoring in Python.” *GitHub*, github.com/giampaolo/psutil. Accessed 15 Sept. 2026.

24. Microsoft. “Sysmon.” *Microsoft Learn: Sysinternals*, learn.microsoft.com/en-us/sysinternals/downloads/sysmon. Accessed 15 Sept. 2026.

25. Liu, Fei Tony, Kai Ming Ting, and Zhi-Hua Zhou. “Isolation Forest.” *Proceedings of the Eighth IEEE International Conference on Data Mining*, IEEE, 2008, pp. 413–422.

26. Breiman, Leo. “Random Forests.” *Machine Learning*, vol. 45, no. 1, 2001, pp. 5–32.

27. Pedregosa, Fabian, et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.

28. Red Canary. “Atomic Red Team.” *GitHub*, github.com/redcanaryco/atomic-red-team. Accessed 15 Sept. 2026.

APPENDIX

A PROJECT STRUCTURE

The project is arranged around monitoring, models, configuration and presentation. Table A.1 gives a guide to the code layout. The source repository remains the authority for exact file names and current implementation details. Keeping collection separate from model evaluation makes it easier to test incomplete observations and to replay recorded feature inputs.

Table A.1 Project Structure

PATH	PURPOSE
agent/collector.py	Collector interfaces
agent/process_monitor.py	Process observations
agent/file_monitor.py	File activity
agent/sentinel_agent.py	Monitoring coordination
twin/digital_twin.py	Behavioural baseline and anomaly model
ai/classifier.py	Random Forest inference
ai/mitre.py	ATT&CK associations
ai/decision_audit.py	Scores, reasons and evidence
core/config.py	Runtime configuration
core/model_store.py	Model loading and recovery
dashboard/	Flask app and browser interface
evaluation/ and tests/	Experiments and automated checks

B INSTALLATION AND DEMONSTRATION

Obtain the project source and follow its README for the supported Python version, dependencies and start-up command. Create a separate virtual environment so that package changes do not affect other applications, and record the source revision and installed package versions before collecting any results.

Configure the watched folder, collector, sampling interval and model locations. Use model files only from the project's trusted workflow and confirm that the feature schema

matches the saved model and baseline. If a new reference is needed, run a known benign workload and let calibration finish before reading normal monitoring scores.

Start the application and open http://127.0.0.1:5000 in a browser. Check the agent state, collector, latest observation time and baseline status. Compare each detector's contribution with the feature explanation, then pause and resume monitoring to confirm that the state changes clearly. Open an evidence panel while new observations arrive, export any incident that must be kept, and check the exported files for sensitive details before sharing them.

C EXPERIMENT RECORD AND TERMS

Each future experiment should have a short record that lets another reviewer understand exactly what was run. Missing measurements should be marked as missing, not replaced by zero. Table C.1 lists the fields to record.

A baseline is the learned reference used for comparison. An observation is one collected set of measurements. A decision holds the detector scores and their interpretation. An alert is a stored decision that meets the alert condition. A technique hypothesis is an ATT&CK association that still needs supporting evidence. Keeping these terms distinct avoids common reporting errors: an alert count is not a count of confirmed attacks, a low anomaly score does not prove the endpoint is safe, and synthetic holdout accuracy is not operational detection quality.

Table C.1 Fields for a Repeatable Experiment

FIELD	WHAT TO RECORD
Run identity	Unique run name, date and purpose
Environment	Operating system, resources and workload
Collection	Collector, target interval and observed timing
Models	Classifier version and baseline state
Configuration	Feature schema, weights and thresholds
Ground truth	Reviewed expected events and labels
Outputs	Telemetry, alerts and exported incidents
Limitations	Missing observations and known gaps